NAVTTC-AI (ML & DL) – Assignment 1

Submission 7 Apr, 2026

Email the submission at iftikharsamee@gmail.com in word file. The word file must contain code and screen shots of results.

Task-1: Basic Python

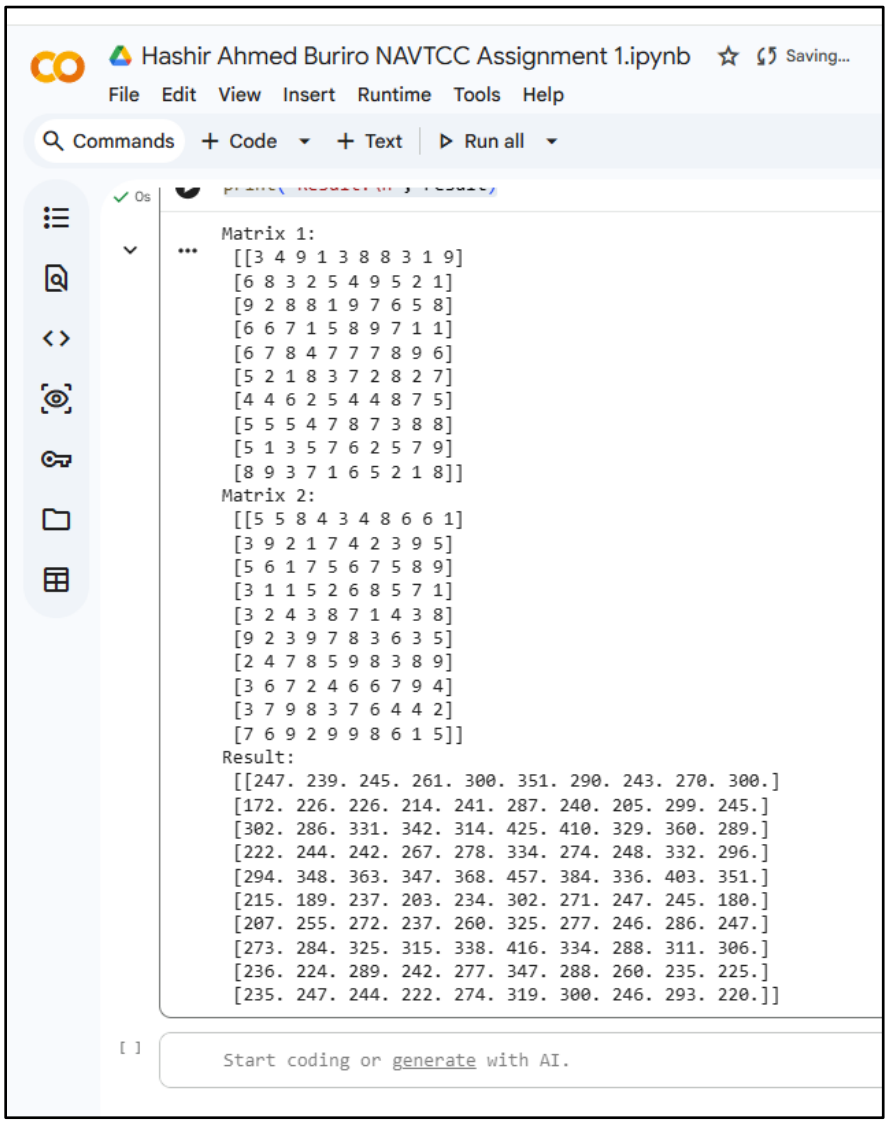
Create a Numpy array of shape (100, 16) with dummy data. Choose 2 slices from this array of size (10,10) each. Using 2 nested for loops, perform matrix multiplication and store the results in new (10,10) array.

CODE:

```
# ==============================
# Task-1: Basic Python
# ==============================
import numpy as np
# Create array of shape (100, 16)
arr = np.random.randint(1, 10, (100, 16))
# Select two slices of size (10,10)
slice1 = arr[0:10, 0:10]
slice2 = arr[10:20, 0:10]
# Initialize result matrix
result = np.zeros((10, 10))
# Matrix multiplication using nested loops
for i in range(10):
    for j in range(10):
        for k in range(10):
            result[i][j] += slice1[i][k] * slice2[k][j]
print("Matrix 1:\n", slice1)
print("Matrix 2:\n", slice2)
print("Result:\n", result)
```

Name: Hashir Ahmed Buriro NAVTTC-AI (ML & DL) Assignment 1 Date: 05-April-2026

OUTPUT:



Task-2: OOP

Create a Class named Calculator and, in this class, create different functions for arithmetic operations like ADD, SUB, DIV, MUL, MOD, PERCENTAGE, as well as Temperature conversion and currency conversion. Take the input from user and return the result**.**

CODE:

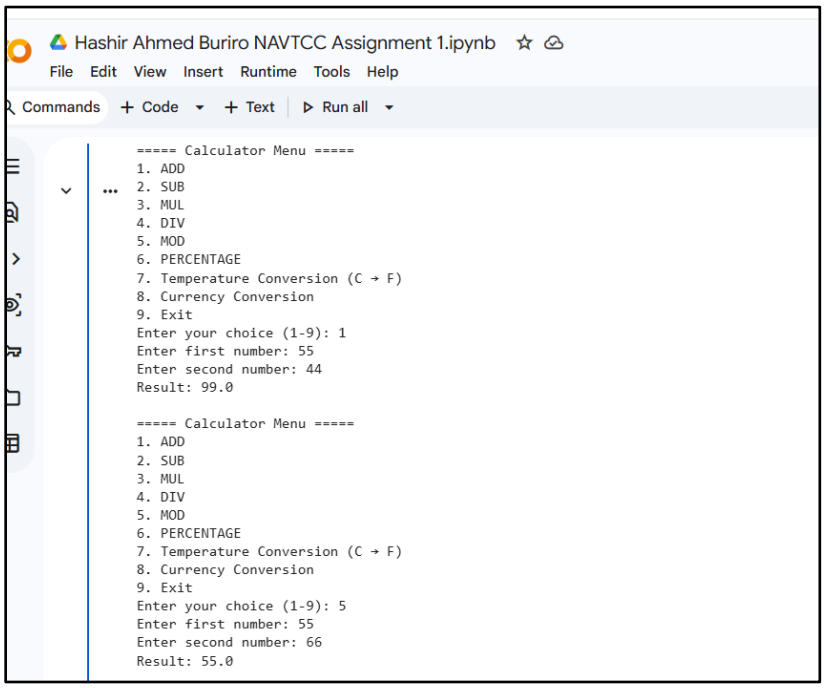
```
# ==============================
# Task-2: OOP
# ==============================
class Calculator:
    def add(self, a, b):
        return a + b
    def sub(self, a, b):
        return a - b
    def mul(self, a, b):
        return a * b
    def div(self, a, b):
        return a / b if b != 0 else "Cannot divide by zero"
    def mod(self, a, b):
        return a % b
    def percentage(self, a, b):
        return (a / b) * 100 if b != 0 else "Invalid"
    def temp_convert(self, c):
        return (c * 9/5) + 32   # Celsius to Fahrenheit
    def currency_convert(self, amount, rate):
        return amount * rate
# Create object
calc = Calculator()
# Loop for continuous execution
while True:
    print("\n===== Calculator Menu =====")
    print("1. ADD")
    print("2. SUB")
    print("3. MUL")
    print("4. DIV")
    print("5. MOD")
    print("6. PERCENTAGE")
    print("7. Temperature Conversion (C → F)")
    print("8. Currency Conversion")
    print("9. Exit")

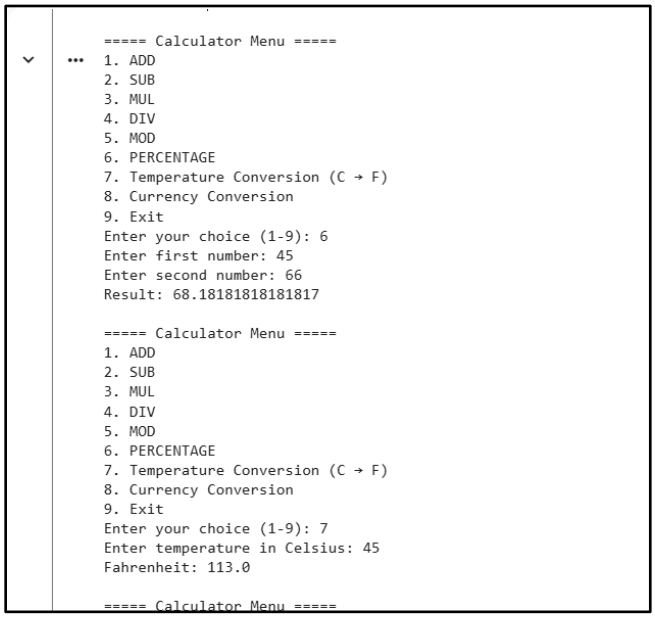
    try:
```

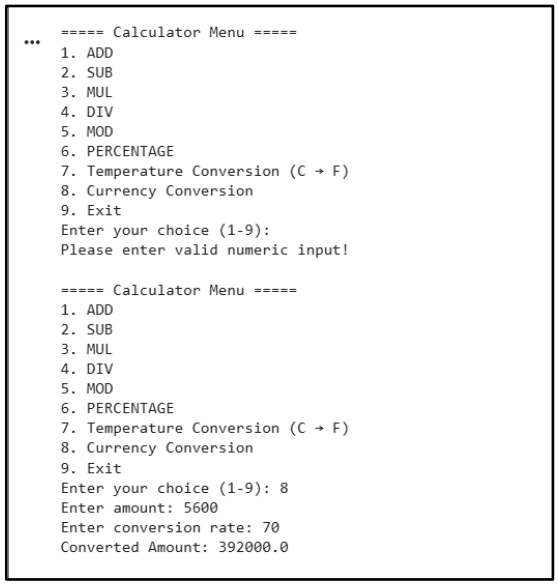
```
        choice = int(input("Enter your choice (1-9): "))
        if choice == 9:
            print("Program Ended 👋")
            break
        # Arithmetic operations
        if choice in [1, 2, 3, 4, 5, 6]:
            a = float(input("Enter first number: "))
            b = float(input("Enter second number: "))
            if choice == 1:
                print("Result:", calc.add(a, b))
            elif choice == 2:
                print("Result:", calc.sub(a, b))
            elif choice == 3:
                print("Result:", calc.mul(a, b))
            elif choice == 4:
                print("Result:", calc.div(a, b))
            elif choice == 5:
                print("Result:", calc.mod(a, b))
            elif choice == 6:
                print("Result:", calc.percentage(a, b))
        # Temperature conversion
        elif choice == 7:
            c = float(input("Enter temperature in Celsius: "))
            print("Fahrenheit:", calc.temp_convert(c))
        # Currency conversion
        elif choice == 8:
            amount = float(input("Enter amount: "))
            rate = float(input("Enter conversion rate: "))
            print("Converted Amount:", calc.currency_convert(amount, rate))
        else:
            print("Invalid choice!")
    except ValueError:
        print("Please enter valid numeric input!")
```

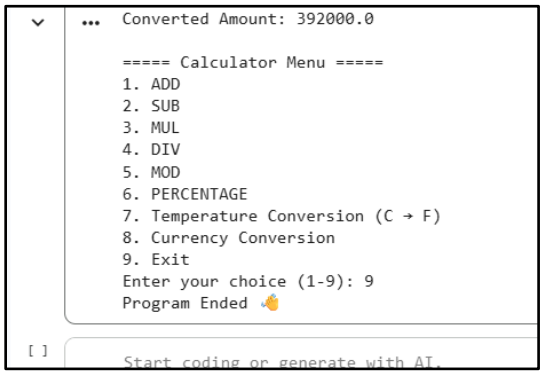
Name: Hashir Ahmed Buriro NAVTTC-AI (ML & DL) Assignment 1 Date: 05-April-2026

OUTPUT:









Task-3: ML-1

Choose 2 classification and 2 regression datasets from internet resources. Preprocess the datasets to convert the string data into numerical (if any column contains string data). Apply linear regression and logistic regression models to get the predictions. You have to split these datasets in 30:70 size for test and train sizes. Finally, show the result of classification result using confusion matrix and also briefly explain in your own words the reflection of result through confusion matrix.

CODE:

```
# ==============================
# Task-3: ML-1
# ==============================

# 1. Import Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.metrics import confusion_matrix, r2_score, mean_squared_error
from sklearn.datasets import load_iris, load_wine, fetch_california_housing,
load_diabetes

# ==============================
# 2. Classification Datasets
# ==============================

# Dataset 1: Iris
iris = load_iris(as_frame=True)
X_iris = iris.data
y_iris = iris.target  # numeric labels

# Dataset 2: Wine
wine = load_wine(as_frame=True)
X_wine = wine.data
y_wine = wine.target  # numeric labels

# Train/Test Split 70/30
X_iris_train, X_iris_test, y_iris_train, y_iris_test = train_test_split(X_iris,
y_iris, test_size=0.3, random_state=42)
X_wine_train, X_wine_test, y_wine_train, y_wine_test = train_test_split(X_wine,
y_wine, test_size=0.3, random_state=42)

# Logistic Regression
logreg_iris = LogisticRegression(max_iter=200)
logreg_iris.fit(X_iris_train, y_iris_train)
y_iris_pred_log = logreg_iris.predict(X_iris_test)
```

```
logreg_wine = LogisticRegression(max_iter=200)
logreg_wine.fit(X_wine_train, y_wine_train)
y_wine_pred_log = logreg_wine.predict(X_wine_test)

# Linear Regression on numeric class labels
linreg_iris = LinearRegression()
linreg_iris.fit(X_iris_train, y_iris_train)
y_iris_pred_lin = linreg_iris.predict(X_iris_test)

linreg_wine = LinearRegression()
linreg_wine.fit(X_wine_train, y_wine_train)
y_wine_pred_lin = linreg_wine.predict(X_wine_test)

# Results
print("=== Iris Dataset ===")
print("Confusion Matrix (Logistic Regression):\n", confusion_matrix(y_iris_test,
y_iris_pred_log))
print("R2 Score (Linear Regression on class labels):", r2_score(y_iris_test,
y_iris_pred_lin))
print("RMSE (Linear Regression on class labels):",
np.sqrt(mean_squared_error(y_iris_test, y_iris_pred_lin)))

print("\n=== Wine Dataset ===")
print("Confusion Matrix (Logistic Regression):\n", confusion_matrix(y_wine_test,
y_wine_pred_log))
print("R2 Score (Linear Regression on class labels):", r2_score(y_wine_test,
y_wine_pred_lin))
print("RMSE (Linear Regression on class labels):",
np.sqrt(mean_squared_error(y_wine_test, y_wine_pred_lin)))

# ==============================
# 3. Regression Datasets
# ==============================

# Dataset 1: California Housing
california = fetch_california_housing(as_frame=True)
X_cal = california.data
y_cal = california.target

# Dataset 2: Diabetes
diabetes = load_diabetes(as_frame=True)
X_dia = diabetes.data
y_dia = diabetes.target

# Train/Test Split 70/30
X_cal_train, X_cal_test, y_cal_train, y_cal_test = train_test_split(X_cal, y_cal,
test_size=0.3, random_state=42)
X_dia_train, X_dia_test, y_dia_train, y_dia_test = train_test_split(X_dia, y_dia,
test_size=0.3, random_state=42)
```

```
# Linear Regression
linreg_cal = LinearRegression()
linreg_cal.fit(X_cal_train, y_cal_train)
y_cal_pred_lin = linreg_cal.predict(X_cal_test)

linreg_dia = LinearRegression()
linreg_dia.fit(X_dia_train, y_dia_train)
y_dia_pred_lin = linreg_dia.predict(X_dia_test)

# Convert numeric target to 3 categories for Logistic Regression
y_cal_cat_train = pd.qcut(y_cal_train, q=3, labels=[0,1,2])
y_cal_cat_test = pd.qcut(y_cal_test, q=3, labels=[0,1,2])

y_dia_cat_train = pd.qcut(y_dia_train, q=3, labels=[0,1,2])
y_dia_cat_test = pd.qcut(y_dia_test, q=3, labels=[0,1,2])

# Logistic Regression
logreg_cal = LogisticRegression(max_iter=500)
logreg_cal.fit(X_cal_train, y_cal_cat_train)
y_cal_pred_log = logreg_cal.predict(X_cal_test)

logreg_dia = LogisticRegression(max_iter=500)
logreg_dia.fit(X_dia_train, y_dia_cat_train)
y_dia_pred_log = logreg_dia.predict(X_dia_test)

# Regression Metrics
r2_cal = r2_score(y_cal_test, y_cal_pred_lin)
rmse_cal = np.sqrt(mean_squared_error(y_cal_test, y_cal_pred_lin))

r2_dia = r2_score(y_dia_test, y_dia_pred_lin)
rmse_dia = np.sqrt(mean_squared_error(y_dia_test, y_dia_pred_lin))

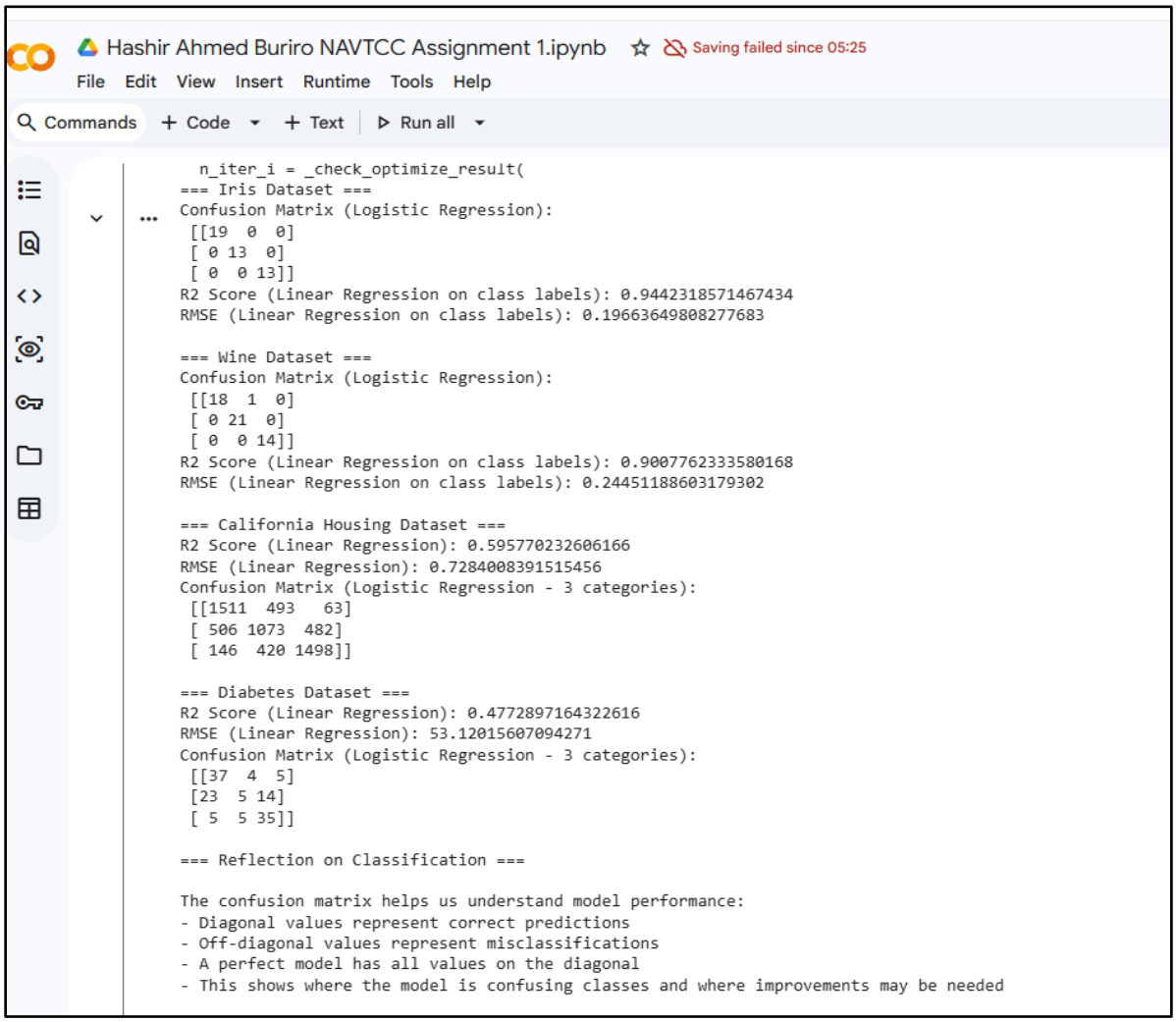
# Print Results
print("\n=== California Housing Dataset ===")
print("R2 Score (Linear Regression):", r2_cal)
print("RMSE (Linear Regression):", rmse_cal)
print("Confusion Matrix (Logistic Regression - 3 categories):\n",
confusion_matrix(y_cal_cat_test, y_cal_pred_log))

print("\n=== Diabetes Dataset ===")
print("R2 Score (Linear Regression):", r2_dia)
print("RMSE (Linear Regression):", rmse_dia)
print("Confusion Matrix (Logistic Regression - 3 categories):\n",
confusion_matrix(y_dia_cat_test, y_dia_pred_log))

# ==============================
# 4. Reflection on Confusion Matrix
# ==============================
```

```
print("\n=== Reflection on Classification ===")
print("""
The confusion matrix helps us understand model performance:
- Diagonal values represent correct predictions
- Off-diagonal values represent misclassifications
- A perfect model has all values on the diagonal
- This shows where the model is confusing classes and where improvements may be
needed
""")
```

OUTPUT:



Explanation:

Reflection on Confusion Matrix:

A confusion matrix shows how well a classification model is performing by comparing the predicted labels to the actual labels.

1. Diagonal values represent correct predictions the higher they are, the better the model is performing.
2. Off-diagonal values represent misclassifications, where the model predicted the wrong class.
3. By analyzing which classes are often confused, we can understand the strengths and weaknesses of the model.
4. For example, if class 1 is often predicted as class 2, the model may have trouble distinguishing features between these two classes.
5. Overall, the confusion matrix provides a visual and numeric way to evaluate classification accuracy and highlights areas for improvement.